

CSC-3TC33

Algorithmique et fondements de l'informatique

Structures de données - partie c

Bertrand Meyer

Automne 2024

Tables de hachage

Table de hachage

But

Structure qui réalise le type abstrait **dictionnaire** et stocke des associations entre clés et valeurs

$$(k_i \mapsto v_i)_{0 \leq i < t}$$

Moyen

Exploiter au mieux les alvéoles d'un tableau T de longueur n , avec $n = \Theta(t)$ et $n > t$, où chaque case $(T[i])_{0 \leq i < n}$ peut contenir un couple (k, v) d'une clé et une valeur.



Fonction de hachage

Définition

Une **fonction de hachage** (non cryptographique) $h : K \rightarrow \mathbb{N}$ associe des données de taille quelconque à des entiers de taille fixe.

Critères souhaités

- Rapide à calculer
- Étale les clés (impression de distribution aléatoire uniforme)

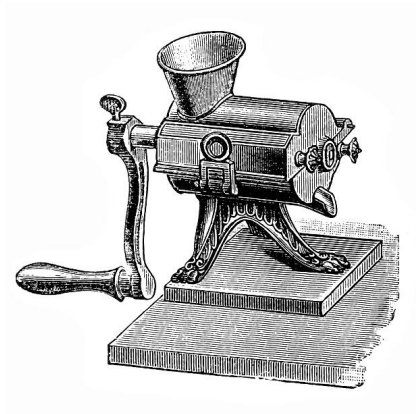


Illustration : la fonction de hachage *SipHash*

Ingrédients

Mélange typique de

- multiplications par des nombres premiers,
- XOR (en python `^`),
- décalages (en python `<<`).

Utilisée par

Python, mais aussi Haskell, Linux, Ruby, Rust, Swift, etc ...

```
def fnv(p):
    if len(p) == 0:
        return 0

    # bit mask, 2**32-1 or 2**64-1
    mask = 2 * sys.maxsize + 1

    x = hashsecret.prefix
    x = (x ^ (ord(p[0]) << 7)) & mask
    for c in p:
        x = ((1000003 * x) ^ ord(c)) & mask
    x = (x ^ len(p)) & mask
    x = (x ^ hashsecret.suffix) & mask

    if x == -1:
        x = -2

    return x
```

(Aumasson & Bernstein 2012)

Implémentation par une table de hachage : illustration



Fonctionnement

On stocke le couple (k, v) en position

$$i = h(k) \bmod n.$$

On recherche l'image de k en sondant T en position $i = h(k) \bmod n$.

Exemple

Clé	Valeur	Haché mod 4
Amine	06.34.56.01.89	1
Sharon	06.59.12.71.30	3
Jürgen	07.75.23.29.94	2

$$h(x) = \text{PremiereLettre}(x)$$

$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

T

← n →

Implémentation par une table de hachage : illustration



Fonctionnement

On stocke le couple (k, v) en position

$$i = h(k) \bmod n.$$

On recherche l'image de k en sondant T en position $i = h(k) \bmod n$.

Exemple

Clé	Valeur	Haché $\bmod 4$
Amine	06.34.56.01.89	1
Sharon	06.59.12.71.30	3
Jürgen	07.75.23.29.94	2

$$h(x) = \text{PremiereLettre}(x)$$

$\emptyset$	Amine	$\emptyset$	$\emptyset$
$\emptyset$	06.34.56.01.89	$\emptyset$	$\emptyset$



Implémentation par une table de hachage : illustration



Fonctionnement

On stocke le couple (k, v) en position

$$i = h(k) \bmod n.$$

On recherche l'image de k en sondant T en position $i = h(k) \bmod n$.

Exemple

Clé	Valeur	Haché $\bmod 4$
Amine	06.34.56.01.89	1
Sharon	06.59.12.71.30	3
Jürgen	07.75.23.29.94	2

$$h(x) = \text{PremiereLettre}(x)$$

$\emptyset$	Amine	$\emptyset$	Sharon
$\emptyset$	06.34.56.01.89	$\emptyset$	06.59.12.71.30



Implémentation par une table de hachage : illustration



Fonctionnement

On stocke le couple (k, v) en position

$$i = h(k) \bmod n.$$

On recherche l'image de k en sondant T en position $i = h(k) \bmod n$.

Exemple

Clé	Valeur	Haché $\bmod 4$
Amine	06.34.56.01.89	1
Sharon	06.59.12.71.30	3
Jürgen	07.75.23.29.94	2

$$h(x) = \text{PremiereLettre}(x)$$

$\emptyset$	Amine	Jürgen	Sharon
$\emptyset$	06.34.56.01.89	07.75.23.29.94	06.59.12.71.30



Facteur de charge

Définition

Le **facteur de charge** est le ratio t/n où t est le nombre d'associations et n la longueur de T .

Redimensionnement

Si le facteur de charge dépasse un seuil, on double la taille de n .

Complexité amortie

Redimensionnement en $\Theta(1)$ pour remplacer tous les éléments déjà insérés.

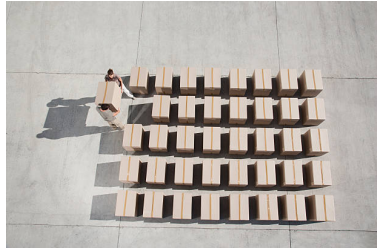


Table de hachage : les collisions

Définition

Une **collision** est le fait de vouloir ranger dans la table deux clés ayant le même haché modulo n .

Exemple

Clé	Valeur	Haché
Amine	06.34.56.01.89	1
Sharon	06.59.12.71.30	3
Jürgen	07.75.23.29.94	2
Sharam	06.83.24.20.72	3

Collision entre Sharon et Sharam.

Résolution collisions par chaînage externe

Principe

On utilise un tableau de listes chaînées.

Exemple

Clé	Valeur	Haché
Amine	06.34.56.01.89	1
Sharon	06.59.12.71.30	3
Jürgen	07.75.23.29.94	2
Sharam	06.83.24.20.72	3



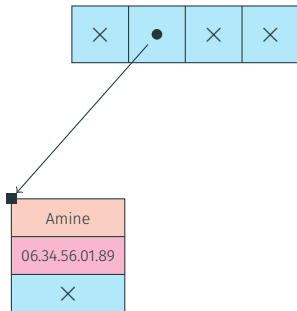
Résolution collisions par chaînage externe

Principe

On utilise un tableau de listes chaînées.

Exemple

Clé	Valeur	Haché
Amine	06.34.56.01.89	1
Sharon	06.59.12.71.30	3
Jürgen	07.75.23.29.94	2
Sharam	06.83.24.20.72	3



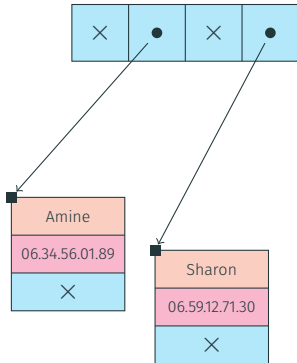
Résolution collisions par chaînage externe

Principe

On utilise un tableau de listes chaînées.

Exemple

Clé	Valeur	Haché
Amine	06.34.56.01.89	1
Sharon	06.59.12.71.30	3
Jürgen	07.75.23.29.94	2
Sharam	06.83.24.20.72	3



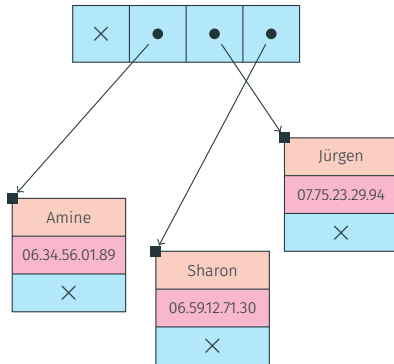
Résolution collisions par chaînage externe

Principe

On utilise un tableau de listes chaînées.

Exemple

Clé	Valeur	Haché
Amine	06.34.56.01.89	1
Sharon	06.59.12.71.30	3
Jürgen	07.75.23.29.94	2
Sharam	06.83.24.20.72	3



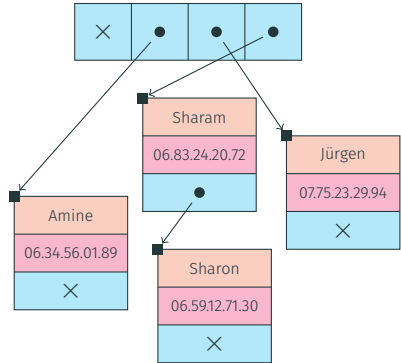
Résolution collisions par chaînage externe

Principe

On utilise un tableau de listes chaînées.

Exemple

Clé	Valeur	Haché
Amine	06.34.56.01.89	1
Sharon	06.59.12.71.30	3
Jürgen	07.75.23.29.94	2
Sharam	06.83.24.20.72	3



Résolution collisions par adressage ouvert

Principe

On envisage d'explorer la table à différentes adresses par sondages successifs jusqu'à rencontrer

- un alvéole libre en cas d'insertion
- la clé en cas de recherche.

Exemple

Clé	Valeur	Haché	Haché 2 ^{aire}	Haché 3 ^{aire}	Etc
Amine	06.34.56.01.89	1	2	3	
Sharon	06.59.12.71.30	3	0	1	
Jürgen	07.75.23.29.94	2	3	0	
Sharam	06.83.24.20.72	3	0	1	

0	0	0	0
0	0	0	0

T[0]

T[1]

T[2]

T[3]

Résolution collisions par adressage ouvert

Principe

On envisage d'explorer la table à différentes adresses par sondages successifs jusqu'à rencontrer

- un alvéole libre en cas d'insertion
- la clé en cas de recherche.

Exemple

Clé	Valeur	Haché	Haché 2 ^{aire}	Haché 3 ^{aire}	Etc
Amine	06.34.56.01.89	1	2	3	
Sharon	06.59.12.71.30	3	0	1	
Jürgen	07.75.23.29.94	2	3	0	
Sharam	06.83.24.20.72	3	0	1	

0	Amine	0	0
0	06.34.56.01.89	0	0

T[0]

T[1]

T[2]

T[3]

Résolution collisions par adressage ouvert

Principe

On envisage d'explorer la table à différentes adresses par sondages successifs jusqu'à rencontrer

- un alvéole libre en cas d'insertion
- la clé en cas de recherche.

Exemple

Clé	Valeur	Haché	Haché 2 ^{aire}	Haché 3 ^{aire}	Etc
Amine	06.34.56.01.89	1	2	3	
Sharon	06.59.12.71.30	3	0	1	
Jürgen	07.75.23.29.94	2	3	0	
Sharam	06.83.24.20.72	3	0	1	

0	Amine	0	Sharon
0	06.34.56.01.89	0	06.59.12.71.30

$\tau[0]$

$\tau[1]$

$\tau[2]$

$\tau[3]$

Résolution collisions par adressage ouvert

Principe

On envisage d'explorer la table à différentes adresses par sondages successifs jusqu'à rencontrer

- un alvéole libre en cas d'insertion
- la clé en cas de recherche.

Exemple

Clé	Valeur	Haché	Haché 2 ^{aire}	Haché 3 ^{aire}	Etc
Amine	06.34.56.01.89	1	2	3	
Sharon	06.59.12.71.30	3	0	1	
Jürgen	07.75.23.29.94	2	3	0	
Sharam	06.83.24.20.72	3	0	1	

0	Amine	Jürgen	Sharon
0	06.34.56.01.89	07.75.23.29.94	06.59.12.71.30

$\tau[0]$

$\tau[1]$

$\tau[2]$

$\tau[3]$

Résolution collisions par adressage ouvert

Principe

On envisage d'explorer la table à différentes adresses par sondages successifs jusqu'à rencontrer

- un alvéole libre en cas d'insertion
- la clé en cas de recherche.

Exemple

Clé	Valeur	Haché	Haché 2 ^{aire}	Haché 3 ^{aire}	Etc
Amine	06.34.56.01.89	1	2	3	
Sharon	06.59.12.71.30	3	0	1	
Jürgen	07.75.23.29.94	2	3	0	
Sharam	06.83.24.20.72	3	0	1	

0	Amine	Jürgen	Sharon
0	06.34.56.01.89	07.75.23.29.94	06.59.12.71.30

T[0]

T[1]

T[2]

T[3]

Résolution collisions par adressage ouvert

Principe

On envisage d'explorer la table à différentes adresses par sondages successifs jusqu'à rencontrer

- un alvéole libre en cas d'insertion
- la clé en cas de recherche.

Exemple

Clé	Valeur	Haché	Haché 2 ^{aire}	Haché 3 ^{aire}	Etc
Amine	06.34.56.01.89	1	2	3	
Sharon	06.59.12.71.30	3	0	1	
Jürgen	07.75.23.29.94	2	3	0	
Sharam	06.83.24.20.72	3	0	1	

Sharam	Amine	Jürgen	Sharon
06.83.24.20.72	06.34.56.01.89	07.75.23.29.94	06.59.12.71.30

$\tau[0]$

$\tau[1]$

$\tau[2]$

$\tau[3]$

Quelques méthodes de sondage usuelles

- Sondage **linéaire** : on se décale d'un pas constant
- Sondage **quadratique** : on se décale d'un pas linéaire
- Hachage **double** : on re-hache le haché
En python : on itère sur les indices

$$i \leftarrow 5i + 1 + \text{perturbation}$$

Files à priorité et tas

Type abstrait de donnée File à priorité

Spécification informelle

Une **file à priorité** est une collection qui permet de stocker des éléments avec une priorité et de les extraire par ordre de priorité (**Plus important, premier servi**).



Type abstrait de donnée File à priorité

Spécification informelle

Une **file à priorité** est une collection qui permet de stocker des éléments avec une priorité et de les extraire par ordre de priorité (**Plus important, premier servi**).

Primitives souhaitées (et leur signature)

- `FàPVide : () → FàP`
- `EstVide : FàP → Booléen`
- `Enfiler : FàP × Elmt × Priorité → FàP`
- `Défiler : FàP → FàP × Elmt × Priorité`



Type abstrait de donnée File à priorité

Spécification informelle

Une **file à priorité** est une collection qui permet de stocker des éléments avec une priorité et de les extraire par ordre de priorité (**Plus important, premier servi**).

Primitives souhaitées (et leur signature)

- $\text{FàPVide} : () \rightarrow \text{FàP}$
- $\text{EstVide} : \text{FàP} \rightarrow \text{Booléen}$
- $\text{Enfiler} : \text{FàP} \times \text{Elmt} \times \text{Priorité} \rightarrow \text{FàP}$
- $\text{Défiler} : \text{FàP} \rightarrow \text{FàP} \times \text{Elmt} \times \text{Priorité}$



Exemples

Accès parallèles à des ressources. Algorithmes de Huffman, Dijkstra, Kruskal (cf. suite du cours).

Spécification formelle du type abstrait File à Priorité

Primitives souhaitées

- $\text{FàPVide} : \{\} \rightarrow \text{FàP}$
- $\text{EstVide} : \text{FàP} \rightarrow \text{Booléen}$
- $\text{Enfiler} : \text{FàP} \times \text{Elmt} \times \text{Priorité} \rightarrow \text{FàP}$
- $\text{Défiler} : \text{FàP} \rightarrow \text{FàP} \times \text{Elmt} \times \text{Priorité}$

Spécification

- La fonction EstVide doit satisfaire à

$$\text{EstVide}(f) = \begin{cases} \text{Vrai} & \text{si } f = \text{FàPVide}() \\ \text{Faux} & \text{si } f \text{ est de la forme } \text{Enfiler}(f', x, p) \end{cases}$$

Spécification formelle du type abstrait File à Priorité

Primitives souhaitées

- $\text{FàPVide} : \{\} \rightarrow \text{FàP}$
- $\text{EstVide} : \text{FàP} \rightarrow \text{Booléen}$
- $\text{Enfiler} : \text{FàP} \times \text{Elmt} \times \text{Priorité} \rightarrow \text{FàP}$
- $\text{Défiler} : \text{FàP} \rightarrow \text{FàP} \times \text{Elmt} \times \text{Priorité}$

Spécification

- Défiler n'est valide que si $\text{EstVide}(f)$ est Faux et alors, en notant $(g, y, q) = \text{Défiler}(f)$

$$\text{Défiler}(\text{Enfiler}(f, x, p)) = \begin{cases} (f, x, p) & \text{si } p > q \\ \text{Enfiler}(g, x, p), y, q & \text{sinon.} \end{cases}$$

Les arbres parfait

Définition

Un arbre binaire est dit **parfait** si toutes les lignes sont remplies, sauf la dernière qui est remplie de gauche à droite.

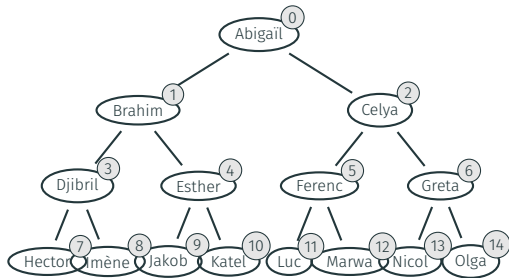
Définition inductive

L'ensemble $\mathcal{P}$ est défini par les règles

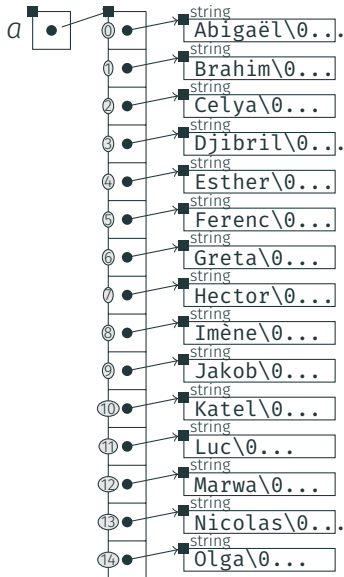
$$\frac{}{\perp \in \mathcal{P}} \qquad \frac{g \in \mathcal{C} \quad d \in \mathcal{P} \quad \text{haut}(g) = \text{haut}(d)}{(e|g|d) \in \mathcal{P}}$$
$$\frac{g \in \mathcal{P} \quad d \in \mathcal{C} \quad \text{haut}(g) = \text{haut}(d) + 1}{(e|g|d) \in \mathcal{P}}$$

où $\mathcal{C}$ désigne l'ensemble des arbres complets.

Représentation des arbres parfaits par un tableau



Le **fils gauche** du sommet i est le sommet $2i + 1$; le **fils droit** est $2i + 2$.
Le **père** est le sommet $(i - 1) // 2$.



Les tas

Définition

Un **tas** est un arbre binaire *parfait* tel que la clé de tout sommet interne est supérieure aux clés de ses fils. Un tas se représente implicitement par un tableau.

Définition inductive

L'ensemble $\mathcal{P}$ est défini par les règles

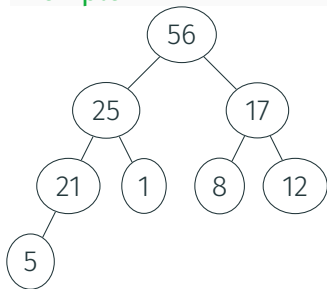
$$\frac{\overline{\perp \in \mathcal{T}} \quad \begin{array}{c} g \in \mathcal{T} \quad d \in \mathcal{T} \quad \left\{ \begin{array}{l} g = \perp \text{ ou } g = (e_g | g_g | d_g) \text{ avec } e_g \leq e \text{ et} \\ d = \perp \text{ ou } d = (e_d | g_d | d_d) \text{ avec } e_d \leq e \end{array} \right. \quad (e | g | d) \in \mathcal{P} \end{array}}{(e | g | d) \in \mathcal{T}}$$

où $\mathcal{P}$ désigne les arbres parfaits.



Structure de tas : exemple

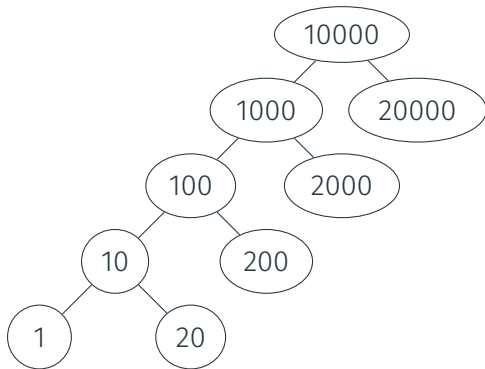
Exemple



Quiz : tas

Vrai ou faux

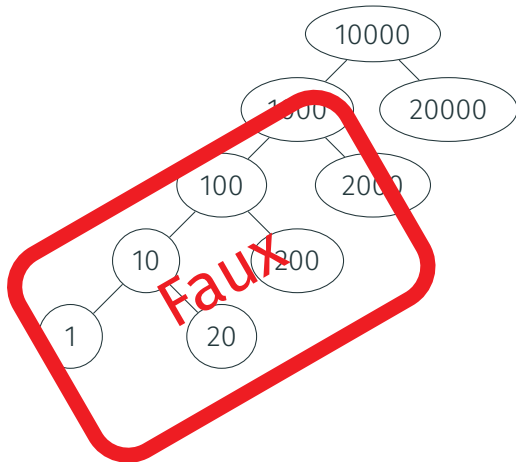
L'arbre suivant est un tas



Quiz : tas

Vrai ou faux

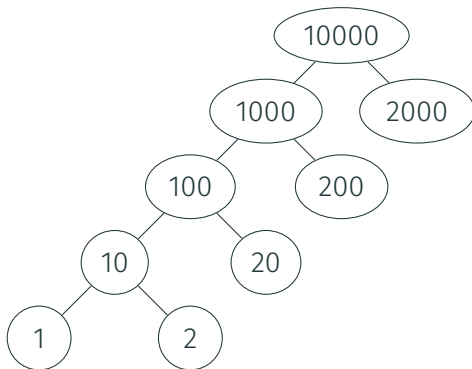
L'arbre suivant est un tas



Quiz : tas

Vrai ou faux

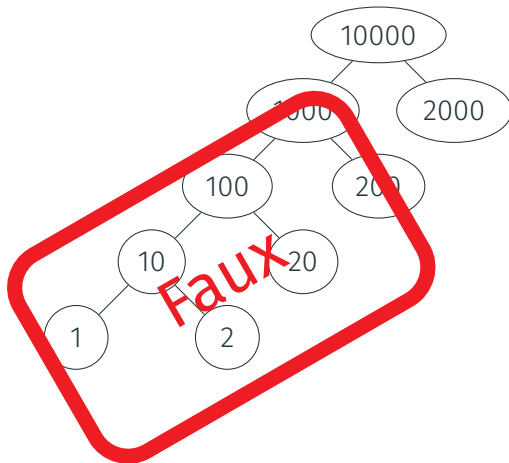
L'arbre suivant est un tas



Quiz : tas

Vrai ou faux

L'arbre suivant est un tas

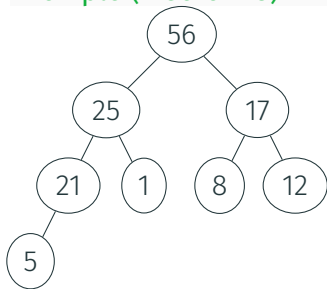


Insertion dans un tas

Principe

- Ajouter l'élément dans le premier alvéole disponible
- Tant que la propriété de tas est violée, échanger l'élément avec son père.

Exemple (Insérer 28)

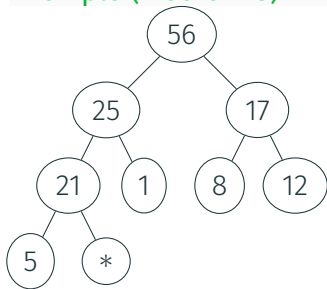


Insertion dans un tas

Principe

- Ajouter l'élément dans le premier alvéole disponible
- Tant que la propriété de tas est violée, échanger l'élément avec son père.

Exemple (Insérer 28)

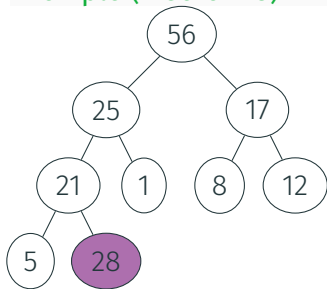


Insertion dans un tas

Principe

- Ajouter l'élément dans le premier alvéole disponible
- Tant que la propriété de tas est violée, échanger l'élément avec son père.

Exemple (Insérer 28)

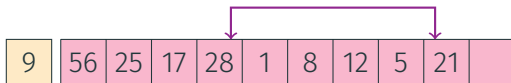
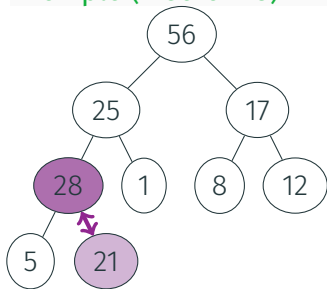


Insertion dans un tas

Principe

- Ajouter l'élément dans le premier alvéole disponible
- Tant que la propriété de tas est violée, échanger l'élément avec son père.

Exemple (Insérer 28)

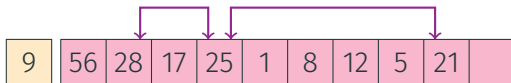
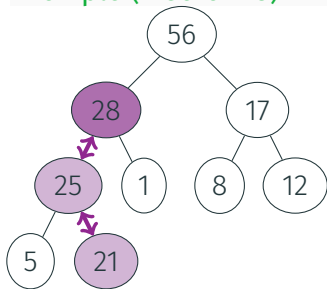


Insertion dans un tas

Principe

- Ajouter l'élément dans le premier alvéole disponible
- Tant que la propriété de tas est violée, échanger l'élément avec son père.

Exemple (Insérer 28)

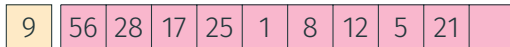
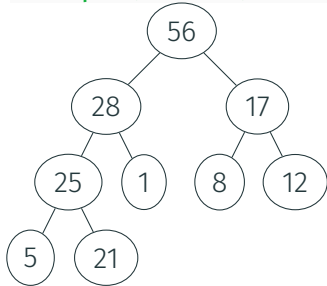


Extraction dans un tas

Principe

- Échanger premier et dernier élément
- Tant que la propriété de tas est violée, échanger l'élément remonté avec son plus grand fils.

Exemple (Extraire)

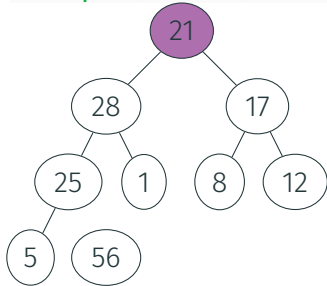


Extraction dans un tas

Principe

- Échanger premier et dernier élément
- Tant que la propriété de tas est violée, échanger l'élément remonté avec son plus grand fils.

Exemple (Extraire)

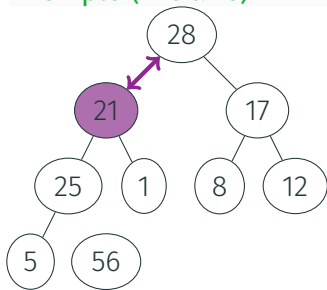


Extraction dans un tas

Principe

- Échanger premier et dernier élément
- Tant que la propriété de tas est violée, échanger l'élément remonté avec son plus grand fils.

Exemple (Extraire)

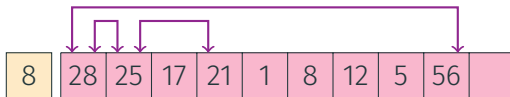
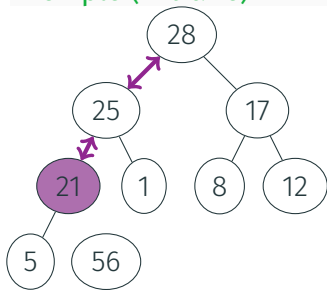


Extraction dans un tas

Principe

- Échanger premier et dernier élément
- Tant que la propriété de tas est violée, échanger l'élément remonté avec son plus grand fils.

Exemple (Extraire)



Module Python

- Python dispose d'un module `heapq`.
- Fonctionne comme des `list` standards, avec les primitives `heapq.heappush` et `heapq.heappop`.
- Cf. documentation pour les détails.

Le tri tas : principe

~1964

(Photo)

Principe

Insérer les éléments successivement dans un tas puis les extraire

Remarque

Le tri peut se faire sur place.

John William Williams
(1930†2012)

Le tri tas : complexité

- La hauteur de l'arbre est en $O(\log n)$ (où n est le nombre d'éléments)
- Les insertions dans le tas et les extractions du tas sont de complexité $O(\log n)$.
- La complexité du tri tas est $O(n \log n)$.

Partitions d'ensembles

Type abstrait de donnée Partition

Spécification informelle

Une **partition** (ou **relation d'équivalence**) permet de représenter la division d'un ensemble en parties disjointes.



Type abstrait de donnée Partition

Spécification informelle

Une **partition** (ou **relation d'équivalence**) permet de représenter la division d'un ensemble en parties disjointes.

Primitives souhaitées (et leur signature)

- Créer : $\text{Ensemble} \rightarrow \text{Partition}$
- Unir : $\text{Partition} \times \text{Élément} \times \text{Élément} \rightarrow \text{Partition}$
- Trouver : $\text{Partition} \times \text{Élément} \rightarrow \text{Identifiant}$



Type abstrait de donnée Partition

Spécification informelle

Une **partition** (ou **relation d'équivalence**) permet de représenter la division d'un ensemble en parties disjointes.

Primitives souhaitées (et leur signature)

- Créer : $\text{Ensemble} \rightarrow \text{Partition}$
- Unir : $\text{Partition} \times \text{Élément} \times \text{Élément} \rightarrow \text{Partition}$
- Trouver : $\text{Partition} \times \text{Élément} \rightarrow \text{Identifiant}$



Exemples

Composantes connexes d'un graphe.

Spécification des primitives

Primitives souhaitées

- Créer : Ensemble $\rightarrow$ Partition
- Unir : Partition $\times$ Clé $\times$ Clé $\rightarrow$ Partition
- Trouver : Partition $\times$ Clé $\rightarrow$ Clé



Spécification

$$\text{Trouver}(\text{Créer}(E), x) = x$$

$$\text{Trouver}(\text{Unir}(P, a, b), a) = \text{Trouver}(\text{Unir}(P, a, b), b)$$

$$\text{Trouver}(\text{Unir}(P, a, b), x) = \begin{cases} \text{Trouver}(\text{Unir}(P, a, b), a) & \text{si} \\ \text{Trouver}(P, x) & \text{ou si} \\ & \text{sinon} \end{cases} \quad \begin{array}{l} \text{Trouver}(P, x) = \text{Trouver}(P, a) \\ \text{Trouver}(P, x) = \text{Trouver}(P, b) \end{array}$$

Implémentation par un tableau

Principe

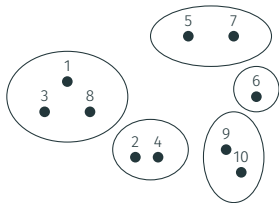
On remplit un tableau avec un n° par composante. Pour fusionner deux composantes, on réécrit les n°s de l'une avec le n° de l'autre.

Implémentation par un tableau

Principe

On remplit un tableau avec un n^o par composante. Pour fusionner deux composantes, on réécrit les n^os de l'une avec le n^o de l'autre.

Exemple

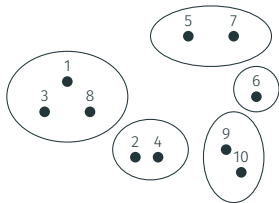


Implémentation par un tableau

Principe

On remplit un tableau avec un n^o par composante. Pour fusionner deux composantes, on réécrit les n^os de l'une avec le n^o de l'autre.

Exemple



3	2	3	2	7	6	7	3	9	9
c[1]	c[2]	c[3]	c[4]	c[5]	c[6]	c[7]	c[8]	c[9]	c[10]

Implémentation naïve et code python

```
class NaivePartition():  
    def __init__(self,n):  
        self.T = [x for x in range(n)]  
    def find(self,x):  
        return self.T[x]  
    def union(self,a,b):  
        for x in range(len(self.T)):  
            if self.T[x] == self.T[b]:  
                self.T[x] = self.T[a]
```

Principe

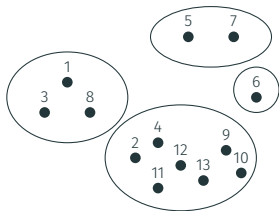
On organise les éléments sous forme de forêts. La composante est désignée par la racine.

Implémentation structure Unir & Trouver

Principe

On organise les éléments sous forme de forêts. La composante est désignée par la racine.

Exemple

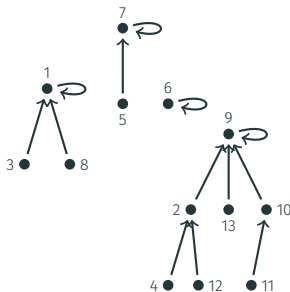
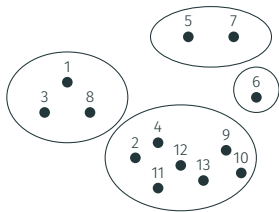


Implémentation structure Unir & Trouver

Principe

On organise les éléments sous forme de forêts. La composante est désignée par la racine.

Exemple



Implémentation de Union-find en python

```
class UnionFind():
    def __init__(self, n):
        self.parent = [i for i in range(n)]
        self.height = [0 for i in range(n)]
    def find(self, u):
        if u == self.parent[u]:
            return(u)
        return self.find(self.parent[u])
    def union(self, u, v):
        p, q = self.find(u), self.find(v)
        if p==q:
            return
        if self.height[p] > self.height[q]:
            self.parent[q] = p
        elif self.height[p] == self.height[q]:
            self.parent[q] = p
            self.height[p] += 1
        else:
            self.parent[p] = q
```

Améliorations

- Retenir la **hauteur des arbres** : unir en plaçant l'arbre le moins haut comme fils de l'arbre le plus haut.
Complexité $O(\log n)$ dans le pire des cas.
- **Comprimer les chemins** : effectuer $\text{père}(v) \leftarrow \text{Trouver}(v)$ à la fin de l'opération $\text{Trouver}(v)$.
Excellente complexité en temps amorti (inverse de la fonction d'Ackerman).